

POSTGRESQL

Isolation Levels

MVCC snapshot-ları, concurrency anomaliyaları və düzgün transaction seçimi

DƏRİN PRAKTİK BƏLƏDÇİ

Read Committed • Repeatable Read • Serializable

Dirty read • Phantom • Lost update • Write skew

EF Core • Retry • Locks • ECafe rezervasiya analizi

Hazırlanma tarixi: 24 sentyabr 2026

1. Isolation nəyi həll edir?

Transaction-lar paralel işləyərkən bir-birinin aralıq və dəyişən vəziyyətini necə görəcəyini müəyyən edir.

Isolation ACID-in "I" hərfidir. Məqsəd bütün əməliyyatları fiziki olaraq bir-birinin ardınca işlətmək deyil; paralelliyi saxlayaraq nəticəni biznes qaydalarına uyğun və proqnozlaşdırılan etməkdir.

Əsas sual

Transaction A işləyərkən Transaction B commit edərsə, A növbəti SELECT-də nə görməlidir? Isolation level bu görünürlük müqaviləsini təyin edir.

Səviyyə	Snapshot sərhədi	PostgreSQL-də əsas nəticə
Read Uncommitted	Standartda ən zəif	Read Committed kimi işləyir
Read Committed	Hər statement üçün yeni snapshot	Eyni transaction-da iki SELECT fərqli nəticə görə bilər
Repeatable Read	İlk real statement-dən etibarən sabit snapshot	Phantom yoxdur; serialization anomaly mümkündür
Serializable	Sabit snapshot + SSI yoxlaması	Nəticə serial icraya ekvivalentdir və konflikt rollback yarada bilər

Daha yüksək isolation həmişə "daha yaxşı" deyil. Daha sabit görünüş və daha sərt düzgünlük əvəzində retry, əlavə monitoring, predicate lock yaddaşı və bəzən aşağı throughput tələb oluna bilər.

2. ACID və transaction sərhədi

Isolation yalnız BEGIN ilə COMMIT/ROLLBACK arasındakı iş vahidinə mənə verir.

Prinsip	Mənası	Isolation-la əlaqəsi
Atomicity	Ya bütün dəyişikliklər commit olur, ya heç biri	Retry bütöv transaction-u təkrar etməlidir
Consistency	Invariant-lar transaction əvvəl/sonra doğrudur	Isolation invariant yoxlamasının köhnəlməsinin qarşısını ala bilər
Isolation	Concurrent əməliyyatların görünülük qaydası	Snapshot, lock və conflict davranışını seçir
Durability	Commit olunmuş nəticə saxlanılır	Isolation-dan ayrılır; WAL/disk siyasəti ilə bağlıdır

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
-- oxu, yoxla, yaz  
COMMIT;
```

Transaction nə qədər böyükdür?

Transaction daxilinə istifadəçi gözləməsi, HTTP çağırışı və fayl upload-u salmaq lock müddətini və konflikt ehtimalını böyüdür. Database kritik zonasını qısa saxlayın.

Savepoint transaction daxilində qismən geri dönüş nöqtəsidir, yeni isolation sərhədi deyil. Eyni transaction snapshot və lock qaydaları davam edir.

3. MVCC necə düşünülməlidir?

PostgreSQL oxucuları və yazıcıları mümkün qədər az bloklamaq üçün sətir versiyalarından istifadə edir.

MVCC - Multi-Version Concurrency Control - bir sətirin zaman üzrə müxtəlif versiyalarını saxlayır. SELECT öz snapshot-una uyğun görünən versiyanı oxuyur; başqa transaction həmin sətiri yeniləsə də köhnə versiya müəyyən müddət əlçatan qalır.

Anlayış	Sadə mental model
Tuple version	UPDATE eyni fiziki məzmunu yerində dəyişmək əvəzinə yeni row version yaradır
xmin / xmax	Versiyanın yaradılması və etibarsızlaşması ilə bağlı transaction identifikatorları
Snapshot	Hansı transaction-ların nəticələrinin görünə biləcəyini müəyyən edən görünüş
VACUUM	Artıq heç bir snapshot-a lazım olmayan köhnə versiyaları təmizləyir

MVCC lock-suz demək deyil

Adi SELECT çox vaxt writer-i bloklamır, amma UPDATE/DELETE, row lock, table lock və schema əməliyyatları yenə gözləmə yarada bilər.

Isolation level snapshot-un nə vaxt alındığını və nə qədər yaşadığını müəyyən edir. Buna görə eyni SQL fərqli isolation level-də fərqli görünürlük və retry davranışı verə bilər.

4. Anomaliyaların xəritəsi

SQL standartının əsas fenomenləri və PostgreSQL-in real zəmanətləri.

Isolation level	Dirty read	Non-repeatable	Phantom	Serialization anomaly
Read Uncommitted (PG)	Yox	Mümkündür	Mümkündür	Mümkündür
Read Committed	Yox	Mümkündür	Mümkündür	Mümkündür
Repeatable Read (PG)	Yox	Yox	Yox	Mümkündür
Serializable	Yox	Yox	Yox	Yox

Standart minimum zəmanətləri müəyyən edir. Database daha güclü davranış verə bilər. PostgreSQL buna iki nümunədir:

- Read Uncommitted istəyi faktiki Read Committed olur; dirty read baş vermir.
- Repeatable Read phantom read-i də dayandırır; standart bunu məcbur etməsə də PostgreSQL daha güclü snapshot verir.

Cədvəl tam hekayə deyil

Lost update və write skew kimi praktik problemlər canonical dörd sətirlik cədvəldə ayrıca sütun deyil. Biznes invariantını qorumaq üçün əməliyyatın real read/write pattern-i ayrıca analiz olunmalıdır.

5. Dirty read

Başqa transaction-un hələ commit etmədiyi məlumatı oxumaq.

```
-- Transaction A          -- Transaction B
UPDATE wallet SET balance=0; SELECT balance FROM wallet;
-- hələ COMMIT yoxdur     -- Ə gərsə dirty read-dir
ROLLBACK;
```

Dirty read təhlükəlidir, çünki oxunan məlumat heç vaxt database tarixçəsinin hissəsi olmaya bilər: yazan transaction rollback edir, amma oxuyan tərəf həmin müvəqqəti dəyərə əsasən e-mail, ödəniş və ya başqa qərar verə bilər.

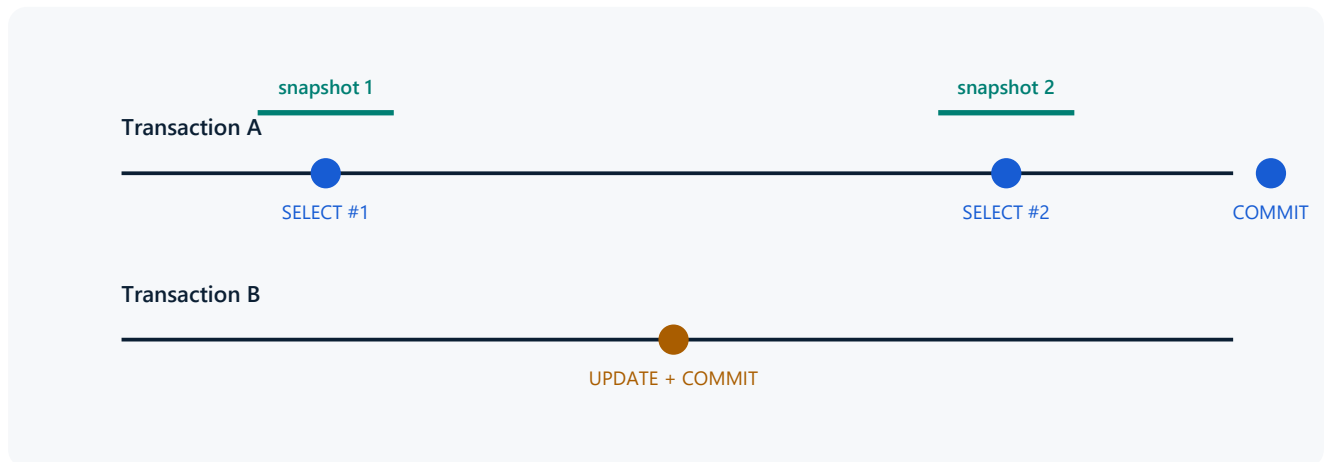
PostgreSQL davranışı

PostgreSQL-də dirty read yoxdur. READ UNCOMMITTED seçsəniz belə sistem READ COMMITTED kimi davranır və yalnız commit olunmuş versiyaları göstərir.

Sual	Cavab
A-nın öz yazısı A-ya görünür?	Bəli, transaction öz commit olunmamış dəyişikliklərini görür
A-nın yazısı B-yə görünür?	Yalnız A commit etdikdən və B-nin snapshot qaydası icazə verdikdən sonra
NOLOCK analoqu varmı?	PostgreSQL MVCC modelində dirty read üçün belə standart rejim yoxdur

6. Non-repeatable read

Eyni transaction eyni row-u ikinci dəfə oxuyanda fərqli commit olunmuş dəyər görür.



Read Committed-də SELECT #1 və SELECT #2 ayrı statement olduqları üçün ayrı snapshot alır. Arada B commit edibsə, A ikinci dəfə yeni dəyəri görə bilər.

Bu həmişə bug deyil

Siyahı ekranı, cari status oxusu və qısa CRUD əməliyyatları üçün ən son commit olunmuş məlumatı görmək faydalıdır. Problem yalnız iki oxunun eyni sabit dünya üzərində hesablanması gözləndikdə yaranır.

Repeatable Read və Serializable transaction daxilində eyni snapshot-u saxlayır. Lakin köhnə snapshot-u görmək "ən son məlumatı" görmək deyil; sabitlik və təzəlik arasında məqsədli seçim edilir.

7. Phantom read

Eyni predicate query ikinci dəfə işləyəndə nəticə dəstinə yeni row daxil olur və ya row çıxır.

```
-- Transaction A / Read Committed
SELECT count(*) FROM reservations
WHERE restaurant_id = 1 AND reserved_at::date = DATE '2026-09-24';
-- nəticə: 4

-- B uyğun 5-ci rezervasiyanı INSERT edib COMMIT edir

SELECT count(*) FROM reservations
WHERE restaurant_id = 1 AND reserved_at::date = DATE '2026-09-24';
-- nəticə: 5 (phantom)
```

Non-repeatable read konkret mövcud row-un dəyişməsidir. Phantom isə predicate-in qaytardığı row dəstinin dəyişməsidir.

PostgreSQL Repeatable Read

Sabit transaction snapshot-u səbəbindən yeni commit olunmuş uyğun row ikinci SELECT-də görünür. Buna görə PostgreSQL Repeatable Read phantom read-i də dayandırır.

Lakin phantom-un görünməməsi biznes invariantının mütləq qorunması demək deyil. İki transaction eyni köhnə snapshot-a əsasən fərqli row-lara yazaraq write skew yarada bilər.

8. Lost update

İki əməliyyat eyni köhnə dəyərdən hesablayıb bir-birinin nəticəsini silə bilər.

```
// Hər iki request balance = 100 oxuyur
var balance = account.Balance;
account.Balance = balance - 30;
await db.SaveChangesAsync();
// Son nəticə 70 ola bilər; iki çıxılmadan biri itib
```

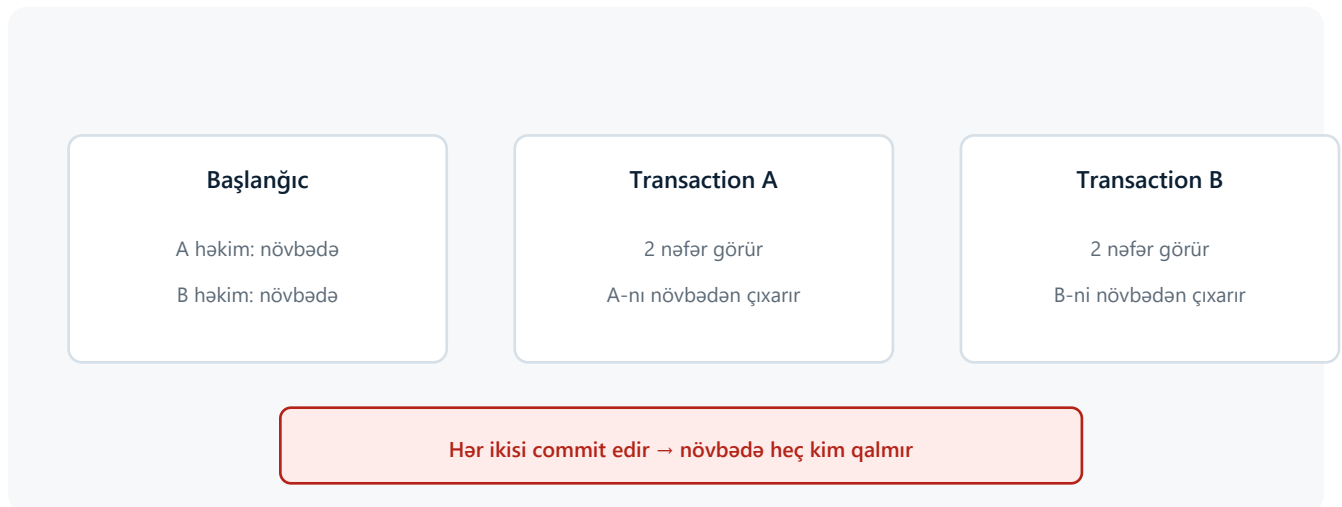
Həll	Nə vaxt seçilir?	Qeyd
Atomic UPDATE	Sadə hesablamalar	UPDATE ... SET balance = balance - 30
SELECT FOR UPDATE	Read-modify-write	Row-u transaction sonuna qədər kilidləyir
Optimistic token / xmin	Konflikt azdır	0 row updated → DbUpdateConcurrencyException
Serializable	Çoxrowlu invariant	40001 retry tələb edir

Isolation level təkbaşına API dizaynı deyil

Read Committed istifadə edərkən atomic SQL, row lock və ya optimistic concurrency token çox vaxt daha konkret və ucuz müdafiədir.

9. Write skew

Transaction-lar fərqli row-lara yazır, amma birlikdə ümumi invariantı pozur.



Hər transaction sabit snapshot-da iki həkimin növbədə olduğunu görür və fərqli həkimi növbədən çıxarır. Eyni row-a yazmadıqları üçün sadə row conflict yoxdur. Hər ikisi commit etdikdə "ən azı bir həkim növbədə olmalıdır" invariantı pozulur.

Repeatable Read niyə kifayət etmir?

Snapshot sabitdir, amma iki transaction-ın oxu-yazı asılılığını serial nəticə kimi yoxlamır. PostgreSQL Serializable SSI bu təhlükəli dependency strukturunu aşkarlayıb bir transaction-u 40001 ilə geri qaytara bilər.

Alternativ həllər: ümumi parent row-u FOR UPDATE ilə kilidləmək, advisory lock ilə invariant açarını serial etmək və ya invariantı constraint formasında database-ə həvalə etmək.

10. Read Uncommitted

SQL standartında mövcuddur, PostgreSQL-də ayrıca real səviyyə deyil.

Standartda Read Uncommitted dirty read, non-repeatable read, phantom və serialization anomaly-yə icazə verə bilər. PostgreSQL MVCC arxitekturasında bu səviyyəni Read Committed kimi xəritələyir.

```
BEGIN TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;  
SHOW transaction_isolation;  
-- PostgreSQL davranışı Read Committed zəmanətlərinə uyğundur
```

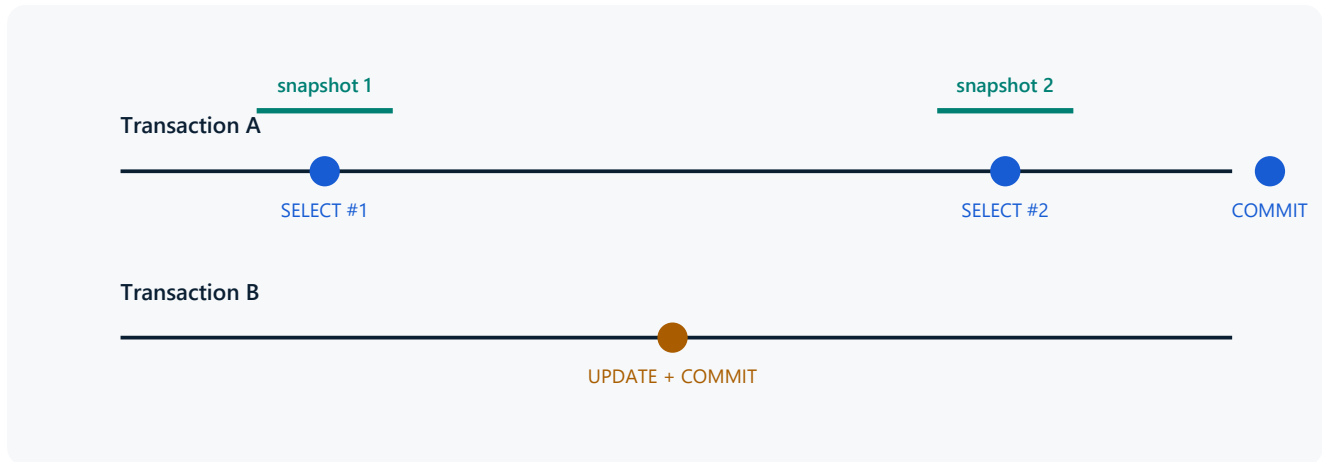
Portativlik xəbərdarlığı

Eyni isolation adı SQL Server, MySQL və PostgreSQL-də tam eyni praktik davranış verməyə bilər. Sənədləşdirmə database engine və versiya ilə birlikdə aparılmalıdır.

PostgreSQL layihəsində Read Uncommitted seçmək performans üçün dirty read açmır. Buna görə kodda bu adı görmək adətən yanlış niyyət və ya başqa database-dən daşınmış düşüncə göstəricisidir.

11. Read Committed

PostgreSQL default-u: hər statement başlananda yeni snapshot.



Bir SELECT icra zamanı sonradan commit olunan dəyişiklikləri görmür; statement daxilində görüntü sabitdir. Lakin növbəti SELECT yeni snapshot aldığı üçün aradakı commit-ləri görə bilər.

Üstünlük	Risk
Yüksək concurrency və az serialization retry	Check-then-act yarışları
Qısa CRUD transaction-ları üçün sadə davranış	Transaction boyu sabit hesabat görüntüsü yoxdur
Lock-dan sonra yeni SELECT son commit-i görə bilir	Bir neçə statement birlikdə avtomatik invariant qorumur

Ən yaxşı istifadə

Qısa request transaction-ları, explicit row/advisory lock, atomic UPDATE və database constraint-lərlə birlikdə Read Committed çox vaxt düzgün default-dur.

12. Read Committed-də UPDATE nüansı

UPDATE gözlədikdən sonra WHERE şərtini yeni row versiyasında yenidən yoxlaya bilər.

UPDATE işləyərkən hədəf row başqa transaction tərəfindən dəyişdirilibsə, PostgreSQL həmin transaction-un bitməsinə gözləyə bilər. Sonra yenilənmiş row versiyasında WHERE şərtini yenidən qiymətləndirir.

```
UPDATE reservations
SET status_id = @confirmed
WHERE id = @id AND status_id = @paymentSubmitted;
```

Əgər başqa transaction status-u artıq dəyişibsə, şərt yeni versiyada false ola və 0 row update oluna bilər. Bu, conditional update-i güclü concurrency primitive edir.

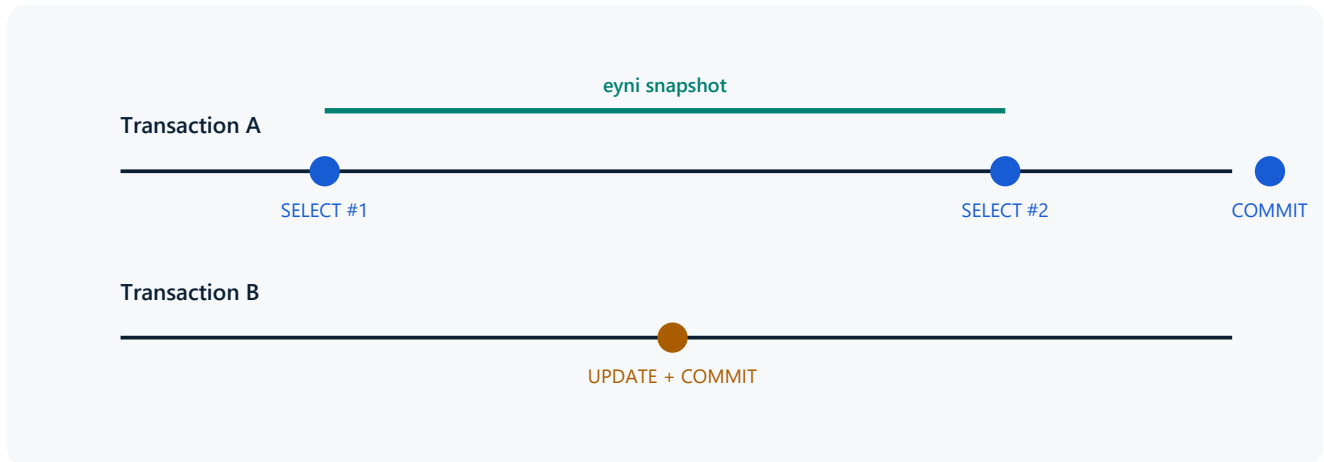
Affected rows yoxlanmalıdır

0 row affected "SQL uğurla işlədi" olsa da biznes baxımından konflikt ola bilər. Tətbiq bunu 409 Conflict və ya uyğun domain error-a çevirməlidir.

Mürəkkəb predicate bir neçə row və aggregate-dən asılıdırsa, tək conditional UPDATE kifayət etməyə bilər; explicit lock, constraint və ya Serializable qiymətləndirilməlidir.

13. Repeatable Read

Transaction daxilində ilk real statement-dən etibarən sabit snapshot.



Eyni transaction daxilində SELECT-lər başqa transaction-ların sonrakı commit-lərini görmür. Öz transaction-unuzun yazıları isə görünür.

Təmin edir	Təmin etmir
Non-repeatable read yoxdur	Bütün biznes invariantlarının serial nəticəsi
PostgreSQL-də phantom yoxdur	Write skew avtomatik dayanmır
Sabit analitik görünüş	Konfliktsiz commit zəmanəti

Retry mümkündür

Concurrent transaction snapshot-dan sonra eyni row-u dəyişibsə, PostgreSQL "could not serialize access due to concurrent update" və SQLSTATE 40001 qaytara bilər. Bütün transaction təkrar edilməlidir.

14. Serializable və SSI

Ən sərt səviyyə uğurlu commit-ləri hər hansı serial icra sırasına ekvivalent edir.

PostgreSQL Serializable, Repeatable Read snapshot davranışına əlavə olaraq Serializable Snapshot Isolation (SSI) ilə transaction-lar arasındakı read/write asılılıqlarını izləyir. Təhlükəli struktur yaranarsa transaction-lardan biri commit əvəzinə geri qaytarılır.

Mexanizm	Davranış
SIREAD / predicate locks	Əvvəlki oxunun nəticəsinə hansı yazının təsir edə biləcəyini izləyir
Blocking	Predicate lock-lar adi lock kimi writer-i bloklamır
Abort	Serial nəticə mümkün deyilsə 40001 serialization_failure
Overhead	Dependency izləmə və predicate lock yaddaşı
Tətbiq öhdəliyi	Bütün transaction-u başdan retry etmək

Serializable sehr deyil

Xarici HTTP side effect artıq göndərilibsə database rollback onu geri qaytarmır. Outbox və idempotency retry-safe dizaynın bir hissəsidir.

Serializable yalnız bütün əlaqəli əməliyyatlar uyğun isolation və transaction sərhədində olduqda güclü zəmanət verir. Transaction xaricindəki qərarlar köhnələ bilər.

15. Read Only Deferrable

Uzun hesabat üçün təhlükəsiz serial snapshot almağın xüsusi yolu.

```
BEGIN TRANSACTION
ISOLATION LEVEL SERIALIZABLE
READ ONLY
DEFERRABLE;

-- uzun hesabat sorğuları
COMMIT;
```

Bu transaction əvvəlcə təhlükəsiz snapshot yaranana qədər gözləyə bilər. Başladıqdan sonra serialization conflict səbəbilə abort edilmə ehtiyacı olmadan sabit, serializable görüntü üzərində oxuya bilər.

Nə vaxt istifadə olunur?

Uzun audit hesabatı, backup-a yaxın consistent export və read-only analitika. Aşağı latency tələb edən adi API GET sorğusu üçün default seçim deyil.

DEFERRABLE seçiminin faydası əsasən SERIALIZABLE + READ ONLY kombinasiyasındadır. Başlanğıc gözləməsi istifadəçi təcrübəsi və job timeout-u ilə uyğunlaşdırılmalıdır.

16. Serialization retry

40001 xəta deyil, protokolun gözlənilən hissəsidir.

PostgreSQL Repeatable Read və Serializable transaction-ları SQLSTATE 40001 ilə dayandıra bilər. Deadlock üçün kod 40P01-dir. Retry yalnız son SQL statement-i deyil, qərar verən bütün transaction funksiyasını təkrar etməlidir.

```
for (var attempt = 1; attempt <= maxAttempts; attempt++)
{
    try
    {
        await ExecuteWholeTransactionAsync(ct);
        break;
    }
    catch (PostgresException ex) when
        (ex.SqlState is "40001" or "40P01")
    {
        await Task.Delay(JitteredBackoff(attempt), ct);
    }
}
```

- Məhdud retry sayı və exponential backoff + jitter istifadə edin.
- Request cancellation və ümumi timeout-a hörmət edin.
- Notification, payment və upload side effect-lərini idempotent/outbox edin.
- Son uğursuzluqda traceId, SQLSTATE, attempt və transaction növünü loglayın.

17. Explicit lock-larla əlaqə

Isolation snapshot qaydasıdır; explicit lock konkret rəqabəti məqsədli serial edir.

Mexanizm	Qoruduğu şey	Tipik istifadə
SELECT FOR UPDATE	Mövcud row-un concurrent update-i	Balans, status transition
Conditional UPDATE	Gözlənilən əvvəlki vəziyyət	Status compare-and-set
Advisory lock	Virtual və ya cross-table biznes açarı	Masa/time slot, customer/day
Table lock	Cədvəl üzrə geniş konflikt	DDL və xüsusi maintenance
Serializable	Ümumi read/write dependency	Mürəkkəb invariant

Read Committed + lock

Lock alındıqdan sonra yeni statement snapshot-u son commit-i görür. Buna görə lock → re-check → write ardıcılığı check-then-act yarışını məqsədli şəkildə həll edə bilər.

Yüksək isolation lock ehtiyacını tam ləğv etmir. Status transition-da FOR UPDATE və ya conditional UPDATE daha aydın ola bilər; Serializable isə write skew kimi geniş anomaliyalara aşkarlayır.

18. Advisory lock ilə fərq

Advisory lock isolation level deyil; tətbiq razılaşması ilə qurulan koordinasiya.

Sual	Isolation level	Advisory lock
Nəyi dəyişir?	Transaction-un görünürlük və conflict modelini	Eyni rəqəmsal açarda əməliyyat sırasını
Avtomatikdirmi?	Transaction-un bütün uyğun statement-lərinə tətbiq olunur	Yalnız lock alan code path-lərə təsir edir
Əsas risk	Retry və köhnə snapshot	Səhv açar və bypass edən kod
ECafe rolu	Read Committed yeni re-check snapshot-u verir	Customer/day və table əməliyyatını serial edir

Birlikdə işləyirlər

ECafe-də `pg_advisory_xact_lock` transaction daxilində alınır. Read Committed lock açıldıqdan sonrakı SELECT-in birinci transaction commit-ini görməsinə imkan verir.

Advisory lock-u Serializable ilə əvəz etmək və ya əksinə etmək avtomatik qərar deyil. Birincisi əvvəlcədən seçilmiş resursu serial edir; ikincisi runtime dependency strukturuna əsasən anomaliyanı aşkarlayır.

19. EF Core / Npgsql tətbiqi

Isolation level transaction başlayarkən explicit seçilə bilər.

```
await using var tx = await db.Database.BeginTransactionAsync(
    IsolationLevel.ReadCommitted,
    cancellationToken);

try
{
    // query, lock, re-check, writes
    await db.SaveChangesAsync(cancellationToken);
    await tx.CommitAsync(cancellationToken);
}
catch
{
    await tx.RollbackAsync(cancellationToken);
    throw;
}
```

EF Core transaction başlamadan hər SaveChanges öz transaction-ını yarada bilər. Bir neçə query və SaveChanges birlikdə atomik olmalıdırsa explicit transaction lazımdır.

Execution strategy

Retrying execution strategy aktivdirsə, manual transaction bütöv retrievable delegate daxilində yaradılmalıdır. Əks halda provider transaction-u tam təkrar edə bilməz.

DbContext-lər arasında eyni transaction paylaşmaq üçün eyni DbConnection və DbTransaction koordinasiyası tələb olunur. Sadəcə iki context-də eyni isolation adı seçmək onları bir transaction etmir.

20. ECafe rezervasiya analizi

Hazırkı ReadCommitted seçimi lock ardıcılığı ilə birlikdə qiymətləndirilməlidir.

Addım	Mexanizm	Təminat
1	Begin ReadCommitted	Qısa request transaction-u
2	Customer advisory xact lock	Eyni restoran/müştəri əməliyyatlarını növbələyir
3	Gündəlik rezervasiya re-check	Əvvəlki transaction commit-ini görür
4	Table advisory xact lock	Eyni masa əməliyyatlarını növbələyir
5	Availability re-check	Double-booking vəziyyətini təzə snapshot-da yoxlayır
6	Insert + history + commit	Atomik yazı və lock-ların avtomatik açılması

Nəticə

Bu flow üçün Read Committed zəif seçim deyil. Düzgün açar, sabit lock sırası və lock-dan sonra re-check sayəsində konkret invariantlar məqsədli qorunur.

Nə vaxt yenidən baxmaq lazımdır?

Bir transaction çoxsaylı masa, sifariş, stok və ödəniş aggregate-lərindən oxuyub fərqli row-lara yazmağa başlayarsa write skew analizi aparılmalı, explicit parent lock və ya Serializable qiymətləndirilməlidir.

21. Ssenariyə görə seçim

Level-i adla deyil, invariant və conflict modelinə görə seçin.

Ssenari	Başlanğıc seçim	Əlavə müdafiə
Sadə CRUD	Read Committed	Constraint və affected rows
Status transition	Read Committed	Conditional UPDATE və ya FOR UPDATE
Masa rezervasiyası	Read Committed	Advisory lock + re-check + constraint
Sabit maliyyə hesabatı	Repeatable Read	Read-only və transaction timeout
Mürəkkəb cross-row invariant	Serializable	40001 full retry
Uzun read-only audit	Serializable Read Only Deferrable	Başlanğıc gözləməsinə limit
High-contention counter	Read Committed	Atomic UPDATE
Optimistic entity editing	Read Committed	xmin/concurrency token

Best practice

Ən aşağı işlək isolation level + ən konkret database primitive + sərt constraint + observability. Lazımsız yüksək isolation həm retry xərci, həm də əməliyyat mürəkkəbliyi yarada bilər.

22. Performans və monitoring

Isolation seçimi latency, conflict və vacuum davranışında görünür.

Metrik	Nəyi göstərir?
Transaction duration	Snapshot və lock-ların nə qədər uzun yaşadığını
40001 sayı	Serialization / concurrent update conflict səviyyəsini
40P01 sayı	Deadlock və lock sırası problemini
Lock wait duration	Blocking contention-i
Idle in transaction	Connection-un açıq transaction ilə boş qalmasını
Dead tuples / vacuum lag	Uzun snapshot-ların köhnə versiya təmizliyinə təsirini
Retry attempts	İstifadəçi latency-si və hotspot-ları

```
SELECT pid, username, state, xact_start, wait_event_type,
       wait_event, left(query, 120) AS query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY xact_start;
```

Uzun transaction təhlükəsi

Sabit snapshot çox uzun qalarsa VACUUM köhnə tuple-ları dərhal təmizləyə bilməz. Bu, table bloat və performans itkisinə səbəb ola bilər.

23. Xüsusi PostgreSQL nüansları

Isolation haqqında testlər bu istisnaları nəzərə almalıdır.

- Sequence dəyişiklikləri transaction rollback ilə geri qaytarılmır və digər transaction-lara dərhal görünə bilər.
- Isolation level ilk query və ya data-modification statement-dən sonra dəyişdirilə bilməz.
- Read Committed-də iki ardıcıl SELECT eyni transaction daxilində fərqli nəticə görə bilər.
- Repeatable Read PostgreSQL-də Snapshot Isolation kimi davranır və phantom read vermir.
- Serializable predicate lock-ları pg_locks görünüşündə SIReadLock kimi görünə bilər.
- Predicate lock-lar yazını bloklamaq üçün deyil, təhlükəli dependency-ni aşkarlamaq üçündür.
- Read-only transaction da Serializable-də dependency analizinə daxil ola bilər; DEFERRABLE təhlükəsiz snapshot üçün xüsusi seçimdir.

Versiya və provider

Davranışı həmişə PostgreSQL versiyası və Npgsql/EF Core provider kontekstində yoxlayın. Başqa database-in eyni adlı isolation level-i ilə avtomatik eyniləşdirməyin.

24. Concurrency test laboratoriyası

İki connection ilə zaman ardıcılığını idarə edən testlər yazın.

Test	A	B	Gözlənilən
Non-repeatable	SELECT, gözlə, SELECT	UPDATE + COMMIT	RC dəyişir; RR dəyişmir
Phantom	COUNT, gözlə, COUNT	INSERT + COMMIT	RC artır; PG RR sabitdir
Lost update	Read-modify-write	Read-modify-write	Token/lock yoxdursa risk
Write skew	Invariant oxu + row A write	Invariant oxu + row B write	RR hər ikisini keçirə bilər; Serializable birini abort edir
FOR UPDATE	Row lock alır və gözləyir	Eyni row update	B A bitənədək gözləyir
Advisory + RC	Lock + check + write	Eyni key lock + check	B A commit-dən sonra yeni vəziyyəti görür

Test qaydaları:

- Eyni DbContext-i paralel task-lar arasında paylaşmayın; iki fiziki connection yaradın.
- Barrier/TaskCompletionSource ilə statement sırasını deterministik edin.
- HTTP nəticəsi ilə yanaşı database-in son vəziyyətini də assert edin.
- 40001 və 40P01 kodlarını ayrıca təsdiqləyin.
- Retry testində side effect-in bir dəfə yarandığını yoxlayın.

25. Dizayn checklist-i

Yeni transaction yazarkən əvvəl bu suallara cavab verin.

☐ Transaction-un qoruduğu invariant nədir?	☐ Hansı row və predicate-lər oxunur, hansılar yazılır?
☐ İki paralel transaction fərqli row-lara yazaraq invariantı poza bilirmi?	☐ Atomic UPDATE və ya constraint kifayətdirmi?
☐ Row/advisory lock açarı və lock sırası müəyyən edilibmi?	☐ Lock-dan sonra re-check edilir?
☐ Serializable seçilibsə 40001 full retry var?	☐ Retry edilən side effect-lər idempotent və ya outbox-dadır?
☐ Transaction daxilində şəbəkə/fayl işi qalıbmı?	☐ Timeout və cancellation siyasəti varmı?
☐ Transaction duration və retry sayı ölçülürmü?	☐ Deterministik concurrency testləri yazılıbmı?

ECafe üçün qısa hökm

Hazırkı rezervasiya flow-unda Read Committed + transaction-scope advisory lock + re-check düzgün kombinasiyadır. Sadəcə isolation level-i artırmaq əvəzinə invariant, key design, lock order, constraint və retry birlikdə idarə olunmalıdır.

26. Terminlər lüğəti

Concurrency mövzusunun əsas sözləri.

Termin	Sadə izah
Snapshot	Transaction-un görə bildiyi commit olunmuş row versiyalarının görünüşü
MVCC	Oxu və yazıları sətir versiyaları ilə paralelləşdirən model
Dirty read	Commit olunmamış başqa transaction məlumatını oxuma
Non-repeatable read	Eyni row-u təkrar oxuyanda fərqli dəyər görmə
Phantom	Eyni predicate query-də row dəstinin dəyişməsi
Lost update	Bir yazının digər concurrent yazının nəticəsini itirməsi
Write skew	Fərqli row yazılarının ümumi invariantı pozması
Serialization failure	Serial nəticə mümkün olmadığı üçün transaction-un 40001 ilə dayandırılması
Predicate lock	Serializable dependency analizində oxunan şərti təmsil edən SReadLock
Optimistic concurrency	Konfliktə bloklamadan, yazı anında versiya/token ilə aşkarlamaq
Pessimistic locking	Konfliktdən əvvəl row/resource lock alaraq növbələndirmək
Idempotency	Təkrar icranın əlavə yan təsir yaratmaması

27. Rəsmi mənbələr

Texniki davranış PostgreSQL, Microsoft EF Core və Npgsql sənədlərinə əsaslanır.

PostgreSQL - Transaction Isolation

<https://www.postgresql.org/docs/current/transaction-iso.html>

Isolation cədvəli, Read Committed, Repeatable Read, Serializable və SSI davranışı.

PostgreSQL - Serialization Failure Handling

<https://www.postgresql.org/docs/current/mvcc-serialization-failure-handling.html>

SQLSTATE 40001, 40P01 və bütün transaction-un retry edilməsi.

PostgreSQL - SET TRANSACTION

<https://www.postgresql.org/docs/current/sql-set-transaction.html>

Isolation, READ ONLY və DEFERRABLE transaction xüsusiyyətləri.

Microsoft - Transactions in EF Core

<https://learn.microsoft.com/ef/core/saving/transactions>

DbContext transaction idarəsi, savepoint və cross-context transaction qaydaları.

Npgsql - Basic Usage / Transactions

<https://www.npgsql.org/doc/basic-usage.html#transactions>

Npgsql transaction API-si, isolation seçimi və savepoint dəstəyi.

ECafe kod istinadları

- ReservationManager - ReadCommitted transaction sərhədləri
- ReservationRepository - customer reservation advisory lock
- TableRepository - table reservation advisory lock

Yekun fikir

Isolation level görünülük müqaviləsidir. Düzgünlük isə isolation, lock, constraint, retry, idempotency və qısa transaction sərhədinin birlikdə işləməsindən yaranır.